

Study Multiple Choice Questions

Study Week 1

Set Product

Assume that the task involved lists instead of sets. What would be the most effective way of changing the solution to the task so that the function does not return an output with duplicate elements?

- a) At the end, remove duplicate elements from the final output.
- b) At the beginning, check if any lists share the same element in the n th position (same index), and remove it from all lists but one.
- c) At the beginning, check each input list for duplicate elements and remove all but one.
- d) (Assuming a recursive solution.) In every recursive call, check whether the input lists contain duplicates and remove them before continuing.

Correct answer: **d)**

Study Week 1

All Permutations

A procedure `nested(int depth)` receives a positive integer `depth`, the number of nested boxes, and shows the order that they should be opened. Boxes are labelled from `depth` (outermost) down to 1 (innermost). For example, the output for `depth = 3` should be:
`\n\n`open 3, open 2, open 1, close 1, close 2, close 3`\n\n`No loops are allowed. Which plan guarantees the correct output and termination for all integers `depth ≥ 1`?

- a) If `depth == 1`, print open 1, close 1. Otherwise, print open `depth`; call `nested(depth - 1)`.
- b) If `depth == 1`, print open 1, close 1 and return. Otherwise, print open `depth`; call `nested(depth - 1)`; then print close `depth`.
- c) If `depth == 0`, return. Otherwise, call `nested(depth - 1)`; then print open `depth`; then print close `depth`.
- d) If `depth == 1`, return an empty string. Otherwise, print open `depth`; call `nested(depth - 1)`; then print close `depth`.

Correct answer: **b)**

Study Week 2

Complex Numbers

Suppose you want to add a function that checks whether a complex number w is a square root of another complex number z (i.e., $w * w = z$). Select the best approach below, choosing the most efficient one if more than one is correct.

- a) Check whether the square of the real component of w equals the real component of z AND the square of the imaginary component of w equals the imaginary component of z .

- b) Check whether the product of the imaginary and real components of w is the square root of the product of the imaginary and real components of z .
- c) Compute w multiplied by w using complex-number multiplication and check whether the result equals z .
- d) None of the above.

Correct answer: c)

Study Week 2

Averager

Suppose we want to change Averager so that it computes and returns the standard deviation of all values added so far, instead of the average. We still want to avoid storing all individual values. Which additional data do we need to store, assuming that we update it on every call to 'add_value'?

- a) Nothing; keeping the current fields 'sum' and 'count' is enough.
- b) Replace 'sum' by a field that stores the previously computed standard deviation and initialize it to 0.
- c) Add a new field that stores the sum of the squares of all added values.
- d) None of the above is sufficient.

Correct answer: c)

Study Week 3

Dynamic Queue

Your Queue class currently maintains two pointers: one to the first node and one to the last node. If you decided to simplify the design by removing the last pointer and only keeping the first pointer, which operation would become significantly slower (less efficient)?

- a) is_empty (checking if the queue is empty)
- b) print (printing the queue in order)
- c) dequeue (removing an element)
- d) enqueue (adding an element)

Correct answer: d)

Study Week 3

Binary Tree Depth

In a valid binary tree, the hierarchy always flows downwards (Parent $\rightarrow$ Child). Suppose you have a valid, minimal solution to this problem., except there was a coding error: a \"leaf\" node's left pointer is accidentally set to point back to the Root node (instead of being nullptr). If you run your get_max_depth function on this \"corrupted\" tree, what will be the result?

- a) It will return the correct depth because the function stops when it hits a node it has already visited.
- b) It will return 0 because the circular path cancels out the depth calculation.
- c) The program will run indefinitely (or crash) because the recursion never reaches a base case.
- d) It will return a very large number representing the total number of nodes in memory.

Correct answer: c)

Study Week 4

Circular Linked List

To compute the length of a linked list, you traverse from the head, counting nodes until reaching null. A student adapts this for circular linked lists: \"Count nodes until the current pointer equals the starting pointer again.\" Assuming a non-empty list, which statement is correct?

- a) The approach fails when the list contains duplicate values.
- b) The approach fails when the list has only one node.
- c) The approach fails when starting from a node other than the head.
- d) The approach does not fail in any of these cases.

Correct answer: d)

Study Week 4

Sorted List

When implementing `remove(value)` for a singly linked sorted list, why must we track the previous node during traversal?

- a) To verify the list remains sorted.
- b) To free the memory of the removed node
- c) To update the pointer that points to the node being removed.
- d) To handle duplicate values correctly

Correct answer: c)

Study Week 5

Array-based Vector

In our `_vector` task, the class owns a dynamically allocated array on the heap. Why is it important to define a custom copy constructor, assignment operator, and destructor?

- a) To guarantee that the memory address of the internal array never changes during the program, even when vectors are copied or assigned.
- b) Because default copy and destruction are only valid for simple built-in types, not for classes like 'tracked'.
- c) To speed up copying by avoiding any heap allocation and letting multiple our_vector objects reuse the same internal array.
- d) To ensure that each our_vector has its own independent copy of the elements and that its heap storage is correctly released when the object is destroyed or reassigned.

Correct answer: **d)**

Study Week 5

First Char

If `first_char(s, 'b')` returns a non-null pointer `p`, how do you compute the index of 'b' in `s`?

- a) `p - s`
- b) `*p - *s`
- c) `&p - &s`
- d) `p[0]`

Correct answer: **a)**

Study Week 6

Polymorphic Functions

After executing `x = y;` in both `f()` and `g()`, which statement about call 3 is correct?

- a) In `f()`, call 3 prints 'B greets John' (polymorphic). In `g()`, call 3 prints 'B greets John' (polymorphic).
- b) In `f()`, call 3 prints 'A says hi to John' (not polymorphic due to slicing). In `g()`, call 3 prints 'B greets John' (polymorphic via `A*`).
- c) In `f()`, call 3 prints 'B greets John' (not polymorphic). In `g()`, call 3 prints 'A says hi to John' (polymorphic).
- d) In both `f()` and `g()`, call 3 prints 'A says hi to Jane'.

Correct answer: **b)**

Study Week 6

House Painting

In the computation `area += (*it)->get_signed_area();`, `Shape::get_signed_area()` calls `get_area()`. Why must `Shape::get_area()` be declared `virtual` (and overridden in derived classes) for the area to reflect rectangles/triangles/circles correctly when stored as `Shape*`?

- a) Because without `virtual`, derived objects cannot be stored in a `std::vector<Shape*>`.
- b) Because without `virtual`, `get_area()` would be bound to `Shape::get_area()` based on the static type `Shape*` (no dynamic dispatch).
- c) Because without `virtual`, `can_paint()` cannot be called from a base class method.
- d) Because without `virtual`, `get_signed_area()` cannot return negative values.

Correct answer: **b)**